

NOTE: This guide explains every single file and code block in the WikiApp project. Read it like a story — from top to bottom — and you will be able to answer any viva question confidently. Every explanation is written assuming you are new to Spring Boot.

Table of Contents

1. The Big Picture — What is Spring Boot and MVC?
2. Project Structure — Where every file lives
3. pom.xml — The shopping list of libraries
4. application.properties — App settings
5. data.sql — Seeding the database
6. WikiAppApplication.java — The entry point
7. Model: LoginForm.java
8. Model: Article.java
9. Model: Admin.java
10. Repository: ArticleRepository.java
11. Repository: AdminRepository.java
12. Service: AuthService.java
13. Service: ArticleService.java
14. Controller: AuthController.java
15. Controller: WikiController.java
16. Controller: AdminController.java
17. Template: login.html
18. Template: wiki/list.html
19. Template: wiki/view.html
20. Template: admin/dashboard.html
21. Template: admin/form.html
22. Template: admin/confirm-delete.html

23. Viva Q&A; — 30 likely questions with answers

1. The Big Picture — Spring Boot & MVC

Before looking at any code, it helps to understand the two main ideas behind this project.

What is Spring Boot?

Spring Boot is a Java framework that makes it easy to build web applications. Normally you would have to install a web server (like Apache Tomcat), write a lot of configuration, and wire hundreds of pieces together manually. Spring Boot does all of that for you automatically. You just write your business logic, run the app, and a working website appears at <http://localhost:8080>.

What is MVC?

MVC stands for Model-View-Controller. It is a way to organise code so that each part has one clear job:

- **Model** — the data (Article, Admin, LoginForm classes)
- **View** — the HTML pages shown to the user (Thymeleaf templates)
- **Controller** — the traffic cop that receives a browser request, asks the Service for data, and picks the right View to display

The three-layer architecture in this project:

- Controller (handles HTTP) → Service (business logic) → Repository (database)
- Each layer only talks to the one directly below it.
- This makes it easy to change one layer without breaking the others.

How a typical request flows through the app:

Step	What happens
1	User opens http://localhost:8080/wiki in the browser
2	Spring routes the GET /wiki request to WikiController.listArticles()
3	WikiController asks ArticleService for all articles
4	ArticleService asks ArticleRepository to run <code>SELECT * FROM article</code>
5	The list of articles travels back up to WikiController
6	WikiController puts them in the Model and returns 'wiki/list'
7	Thymeleaf fills wiki/list.html with the data and sends HTML to the browser

2. Project Structure

Every file in the project has a specific home. Spring Boot expects this structure and will not find your files if they are in the wrong place.

Project folder layout

```
WikiApp/

pom.xml <-- Maven build file (libraries list)

src/main/java/com/wikiapp/

WikiAppApplication.java <-- Entry point, starts the server

model/

LoginForm.java <-- Form data object (Part A)

Article.java <-- Database table: ARTICLE

Admin.java <-- Database table: ADMIN

repository/

ArticleRepository.java <-- Database queries for articles

AdminRepository.java <-- Database queries for admins

service/

AuthService.java <-- Login business logic

ArticleService.java <-- Article business logic

controller/

AuthController.java <-- Handles /login and /logout

WikiController.java <-- Handles /wiki (public)

AdminController.java <-- Handles /admin (protected)

src/main/resources/

application.properties <-- App settings (port, database)

data.sql <-- Seed data run at startup

templates/

login.html, welcome.html, error.html

wiki/list.html, wiki/view.html

admin/dashboard.html, admin/form.html, admin/confirm-delete.html

static/css/style.css <-- Stylesheet
```

3. pom.xml — The Library Shopping List

Maven is the build tool. pom.xml is the configuration file that tells Maven which libraries to download from the internet and include in the project. You never have to download JAR files manually.

The five key dependencies:

Dependency	What it gives us
spring-boot-starter-web	Embedded Tomcat server, @Controller, @GetMapping, @PostMapping
spring-boot-starter-thymeleaf	Thymeleaf template engine — turns .html files into dynamic pages
spring-boot-starter-validation	@NotBlank and other form validation annotations
spring-boot-starter-data-jpa	JPA / Hibernate — talks to the database; JpaRepository
h2	H2 in-memory/file database — no install needed, stores data in ./data/wikidb

TIP: In the viva, if asked 'why do you use H2?', say: H2 is an embedded database — it runs inside the Java process, needs no installation, and stores data in a file so it survives restarts. Perfect for development and assessment.

4. application.properties — App Settings

This file configures how the application runs. Spring Boot reads it at startup. Each line is a key=value pair.

```
src/main/resources/application.properties

server.port=8080

spring.thymeleaf.cache=false

spring.application.name=WikiApp


spring.datasource.url=jdbc:h2:file:./data/wikidb;AUTO_SERVER=TRUE

spring.datasource.driver-class-name=org.h2.Driver

spring.datasource.username=sa

spring.datasource.password=


spring.h2.console.enabled=true

spring.h2.console.path=/h2-console


spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=false

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect


spring.jpa.defer-datasource-initialization=true

spring.sql.init.mode=always
```

Setting	Plain-English explanation
server.port=8080	The web server listens on port 8080, so the URL is localhost:8080
thymeleaf.cache=false	HTML changes show immediately without restarting the server
datasource.url=jdbc:h2:file:./data/wikidb	Store the database in the ./data/ folder as a file (survives restarts)
AUTO_SERVER=TRUE	Allows the H2 console and the app to share the same database file simultaneously
h2.console.enabled=true	Enables a browser-based database viewer at /h2-console
ddl-auto=update	JPA creates tables if they don't exist, updates them if the entity changes. Never drops data.
defer-datasource-initialization=true	Runs data.sql AFTER JPA has created the tables (important for seeding)
sql.init.mode=always	Runs data.sql on every startup, not just the first one

5. data.sql — Seeding the Database

Spring runs this SQL file automatically every time the application starts. It pre-fills the database with one admin account and five sample articles so the wiki has content to show straight away.

```
src/main/resources/data.sql (simplified)

MERGE INTO ADMIN (id, username, password) KEY(id)
VALUES (1, 'movindu', '123');

MERGE INTO ARTICLE (id, title, category, content, last_updated) KEY(id) VALUES
(1, 'Welcome to WikiApp', 'General', 'This is a small Wiki...', CURRENT_TIMESTAMP);
... (4 more articles) ...

ALTER TABLE ARTICLE ALTER COLUMN ID RESTART WITH 100;
ALTER TABLE ADMIN ALTER COLUMN ID RESTART WITH 100;
```

Why MERGE instead of INSERT?

INSERT would fail on the second startup because the rows already exist. MERGE means: 'insert this row if it does not exist yet, otherwise leave it alone'. It is safe to run every startup.

Why ALTER COLUMN ID RESTART WITH 100?

The seeded articles use IDs 1 to 5. If the auto-increment counter also starts at 1, the first new article you create would try to use ID 1 and crash with a duplicate-key error. Restarting the counter at 100 means new rows get IDs 100, 101, 102 ... which never collide with the seeded ones.

6. WikiAppApplication.java — The Entry Point

```
src/main/java/com/wikiapp/WikiAppApplication.java

@SpringBootApplication

public class WikiAppApplication {

    public static void main(String[] args) {

        SpringApplication.run(WikiAppApplication.class, args);

    }

}
```

Line by line:

@SpringBootApplication — One annotation that does three things at once: (1) marks this as a configuration class, (2) tells Spring to scan the com.wikiapp package for Controllers, Services and Repositories, (3) enables auto-configuration (Spring guesses the right settings based on the libraries in pom.xml).

SpringApplication.run(...) — Hands control to Spring Boot. Spring then starts the embedded Tomcat web server, connects to the H2 database, creates the JPA tables, runs data.sql, and wires all the beans (Controllers, Services, Repositories) together. After this one line finishes, the app is fully running.

TIP: If asked 'what does @SpringBootApplication do?' — say it triggers component scanning (finds all your @Controller, @Service, @Repository classes automatically) and enables auto-configuration (sets up the database, Thymeleaf, validation with no extra code).

7. Model: LoginForm.java

LoginForm is a simple Java class (called a POJO — Plain Old Java Object) that holds the username and password the user typed into the login form. It is NOT stored in the database — it only exists during a single request.

```
src/main/java/com/wikiapp/model/LoginForm.java

public class LoginForm {

    @NotBlank(message = "Username is required")
    private String username;

    @NotBlank(message = "Password is required")
    private String password;

    public LoginForm() { } // Spring needs this to create an empty object

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

@NotBlank

This annotation tells Spring's validation system: 'reject this form if the field is null, empty, or only spaces'. If validation fails, Spring fills a BindingResult object with the error messages and the Controller can re-display the form.

Getters and setters

Spring uses the setter (setUsername) to copy the form field value into the object. Thymeleaf uses the getter (getUsername) to read the value back when re-displaying the form. Without them, the binding would silently fail.

8. Model: Article.java

Article is a JPA entity — a Java class that is automatically mapped to a database table called ARTICLE. Every field becomes a column. Every instance becomes a row.

```
src/main/java/com/wikiapp/model/Article.java

@Entity

public class Article {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Title is required")
    @Column(nullable = false)
    private String title;

    @NotBlank(message = "Category is required")
    @Column(nullable = false)
    private String category;

    @NotBlank(message = "Content is required")
    @Column(nullable = false, length = 5000)
    private String content;

    private LocalDateTime lastUpdated;

    // No-args constructor required by JPA
    public Article() { }

    // Getters and setters for all fields ...
}
```

Annotation	What it means in plain English
@Entity	Maps this Java class to a database table (JPA creates the table for you)
@Id	Marks the primary key — the unique identifier for each row
@GeneratedValue(IDENTITY)	The database auto-increments the ID — you never set it manually
@Column(nullable=false)	The database column cannot be NULL — enforced at the DB level too

@Column(length=5000)	Sets the VARCHAR column size to 5000 characters for long article bodies
@NotBlank	Validates the field in the form before saving — shows an error if empty
LocalDateTime lastUpdated	Not annotated separately — JPA creates a TIMESTAMP column automatically

9. Model: Admin.java

Admin is the other JPA entity — it maps to the ADMIN table. In Part A the credentials were hard-coded in Java. In Part B they live in the database, which means you can add more admins without touching the code.

```
src/main/java/com/wikiapp/model/Admin.java

@Entity

public class Admin {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password; // plain text (acceptable for this assessment)

    public Admin() { }

    // Getters and setters ...

}
```

unique = true on username

This creates a UNIQUE constraint in the database so two admins cannot share the same username. The database enforces this even if the Java code accidentally tries to insert a duplicate.

TIP: If asked 'why store passwords in plain text?', say: For simplicity and ease of explanation in this assessment. In a real production application we would always hash passwords using BCrypt so that even if the database is stolen, the passwords cannot be read.

10. Repository: ArticleRepository.java

The Repository layer is the only part of the code that talks to the database. We never write SQL — instead we extend Spring's JpaRepository interface and Spring generates all the SQL at startup.

```
src/main/java/com/wikiapp/repository/ArticleRepository.java

public interface ArticleRepository extends JpaRepository<Article, Long> {

    List<Article> findByCategoryIgnoreCase(String category);

    List<Article> findByTitleContainingIgnoreCase(String text);

}
```

JpaRepository<Article, Long>

The two type parameters mean: 'the entity is Article, the primary key type is Long'. By extending this interface we automatically get these methods for free — no code needed:

- findAll() — SELECT * FROM article
- findById(id) — SELECT * FROM article WHERE id = ?
- save(article) — INSERT or UPDATE (JPA decides based on whether id is set)
- deleteById(id) — DELETE FROM article WHERE id = ?
- existsById(id) — SELECT COUNT(*) FROM article WHERE id = ?

Custom query methods — naming convention magic

Spring reads the method name and generates the SQL automatically. findByCategoryIgnoreCase(String category) becomes:

Generated SQL

```
SELECT * FROM article WHERE UPPER(category) = UPPER(?)
```

findByTitleContainingIgnoreCase(String text) becomes:

Generated SQL

```
SELECT * FROM article WHERE UPPER(title) LIKE UPPER('%' || ? || '%')
```

11. Repository: AdminRepository.java

```
src/main/java/com/wikiapp/repository/AdminRepository.java

public interface AdminRepository extends JpaRepository<Admin, Long> {

    Optional<Admin> findByUsername(String username);

}
```

Why Optional<Admin> instead of Admin?

If the username does not exist in the database, returning plain Admin would give us null, and any code that forgets to null-check would crash with a NullPointerException. Optional forces the caller to handle both cases — found and not-found — without any null checking. We call maybeAdmin.isEmpty() to check if nothing was found, and maybeAdmin.get() to retrieve the actual Admin object.

NOTE: Think of Optional like a gift box. The box is always there (never null). Sometimes there is a present inside (the Admin), sometimes the box is empty. You check the box before reaching in.

12. Service: AuthService.java

The Service layer holds business logic. AuthService decides whether a given username/password combination is valid. The Controller does not know HOW authentication works — it just calls `isAuthenticated()` and gets a yes or no.

```
src/main/java/com/wikiapp/service/AuthService.java

@Service

public class AuthService {

    private final AdminRepository adminRepository;

    @Autowired

    public AuthService(AdminRepository adminRepository) {
        this.adminRepository = adminRepository;
    }

    public boolean isAuthenticated(LoginForm form) {
        if (form == null) return false;

        String username = form.getUsername();
        String password = form.getPassword();

        if (username == null || username.trim().isEmpty()) return false;
        if (password == null || password.trim().isEmpty()) return false;

        // Normalise: 'Movindu' and 'movindu' both work
        String cleanedUsername = username.trim().toLowerCase();

        Optional<Admin> maybeAdmin = adminRepository.findByUsername(cleanedUsername);
        if (maybeAdmin.isEmpty()) return false;

        Admin admin = maybeAdmin.get();

        return admin.getPassword().equals(password); // .equals() compares text
    }
}
```

@Service

Marks this class as a Spring-managed bean. Spring creates one instance and injects it wherever it is needed (e.g. into AuthController).

@Autowired constructor injection

Spring sees that AuthService needs an AdminRepository, finds the one it already created, and passes it in through the constructor automatically. This is called Dependency Injection — we do not call new AdminRepository() ourselves.

Why .equals() and not ==?

In Java, == on String objects compares memory addresses, not content. Two different String objects with the same text would return false with ==. .equals() compares the actual characters, which is what we want.

13. Service: ArticleService.java

ArticleService is the go-between for article data. The Controller asks it to find/save/delete articles. The Service adds any extra logic (like stamping the lastUpdated time) before delegating to the Repository.

src/main/java/com/wikiapp/service/ArticleService.java (key methods)

```
public List<Article> findAll() {
    return articleRepository.findAll();
}

public Optional<Article> findById(Long id) {
    return articleRepository.findById(id);
}

public List<Article> findByCategory(String category) {
    if (category == null || category.trim().isEmpty()) return findAll();
    return articleRepository.findByCategoryIgnoreCase(category.trim());
}

public List<Article> search(String text) {
    if (text == null || text.trim().isEmpty()) return findAll();
    return articleRepository.findByTitleContainingIgnoreCase(text.trim());
}

public Article save(Article article) {
    article.setLastUpdated(LocalDateTime.now()); // always stamp the time
    return articleRepository.save(article);
}

public void deleteById(Long id) { articleRepository.deleteById(id); }
public boolean existsById(Long id) { return articleRepository.existsById(id); }
```

Why does save() work for both CREATE and UPDATE?

JPA's save() checks whether the Article has an id set. If the id is null (new article) — it runs INSERT. If the id is already set (existing article) — it runs UPDATE. In AdminController we call article.setId(id) before saving an edit, which tells JPA to update the existing row.

14. Controller: AuthController.java

AuthController handles login, logout and the register placeholder. It is the first controller the user interacts with.

```
src/main/java/com/wikiapp/controller/AuthController.java

@Controller
@RequestMapping("/")
public class AuthController {

    public static final String SESSION_ADMIN_KEY = "loggedInAdmin";

    private final AuthService authService;

    @Autowired
    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @GetMapping({"", "/login"})
    public String showLoginPage(Model model) {
        model.addAttribute("loginForm", new LoginForm());
        return "login";
    }

    @PostMapping("/login")
    public String processLogin(@ModelAttribute LoginForm loginForm,
        HttpSession session, Model model) {
        boolean ok = authService.isAuthenticated(loginForm);
        if (ok) {
            session.setAttribute(SESSION_ADMIN_KEY, loginForm.getUsername().trim().toLowerCase());
            return "redirect:/admin";
        }
        model.addAttribute("errorMessage", "Invalid username or password.");
        return "error";
    }

    @GetMapping("/logout")
```

```

public String logout(HttpSession session) {
    session.invalidate();
    return "redirect:/login";
}
}

```

Code	Plain-English meaning
@Controller	Marks this class as a Spring MVC controller (handles HTTP requests)
@RequestMapping("/")	All URLs in this class start from the root path /
SESSION_ADMIN_KEY = "loggedInAdmin"	A constant string used as the key when storing the username in the session
@GetMapping({"/", "/login"})	Respond to GET requests for both / and /login with the same method
model.addAttribute("loginForm", new LoginForm());	Put an empty LoginForm in the Model so Thymeleaf can bind the form fields to it
@PostMapping("/login")	Respond to the form submission (browser sends POST when user clicks Log in)
@ModelAttribute LoginForm	Spring reads the posted form fields and fills them into a LoginForm object
session.setAttribute(...)	Store the admin's username in the session — like a sticky note on the browser
return "redirect:/admin"	Tell the browser to navigate to /admin (Post-Redirect-Get pattern)
session.invalidate()	Delete everything in the session — the user is now logged out

TIP: If asked 'what is a session?': a session is server-side storage tied to one browser. Spring gives each browser a unique cookie (JSESSIONID). When the browser sends a request, Spring looks up that cookie and retrieves any data we stored with setAttribute. When we call invalidate(), that data is deleted and the user must log in again.

15. Controller: WikiController.java

WikiController handles the public-facing wiki pages. Anyone can access these — no login required. It also passes an isAdmin flag to the templates so they can show extra buttons when an admin is logged in.

src/main/java/com/wikiapp/controller/WikiController.java (key parts)

```
@Controller

@RequestMapping("/wiki")

public class WikiController {

    private boolean isAdmin(HttpSession session) {

        return session.getAttribute(AuthController.SESSION_ADMIN_KEY) != null;

    }

    @GetMapping

    public String listArticles(

        @RequestParam(required = false) String category,

        @RequestParam(required = false) String search,

        HttpSession session, Model model) {

        List<Article> articles;

        if (search != null && !search.trim().isEmpty())

            articles = articleService.search(search);

        else if (category != null && !category.trim().isEmpty())

            articles = articleService.findByCategory(category);

        else

            articles = articleService.findAll();

        // Build a sorted list of unique category names for the filter buttons.

        // Loop through every article; if the category is not already in the

        // list, add it. Then sort the list alphabetically.

        List<String> categories = new ArrayList<>();

        for (Article a : articleService.findAll()) {

            if (!categories.contains(a.getCategory())) {

                categories.add(a.getCategory());

            }

        }

    }

}
```

```

}

Collections.sort(categories);

model.addAttribute("articles", articles);
model.addAttribute("categories", categories);
model.addAttribute("category", category);
model.addAttribute("search", search);
model.addAttribute("isAdmin", isAdmin(session));
return "wiki/list";
}

@GetMapping("/{id}")
public String viewArticle(@PathVariable Long id,
    HttpSession session, Model model) {

    Optional<Article> maybeArticle = articleService.findById(id);
    if (maybeArticle.isEmpty()) {
        model.addAttribute("errorMessage", "Article not found.");
        return "error";
    }

    model.addAttribute("article", maybeArticle.get());
    model.addAttribute("isAdmin", isAdmin(session));
    return "wiki/view";
}
}

```

@RequestParam(required = false)

URL parameters like `/wiki?search=java` or `/wiki?category=Programming` are optional. If the URL does not include them, Spring passes null instead of crashing.

How the categories list is built

We start with an empty `ArrayList` called `categories`. We loop through every article with a `for` loop. For each article, we check: is this article's category already in the list? If not, we add it. After the loop, `Collections.sort()` sorts the list alphabetically. This gives us e.g. `['General', 'History', 'Programming', 'Testing']` which the template uses to display the clickable filter pills.

16. Controller: AdminController.java

AdminController is the most complex controller. It handles the full CRUD cycle for articles and protects every method with a session check.

```
src/main/java/com/wikiapp/controller/AdminController.java (structure)

@Controller
@RequestMapping("/admin")
public class AdminController {

    private boolean isLoggedIn(HttpSession session) {
        return session.getAttribute(AuthController.SESSION_ADMIN_KEY) != null;
    }

    // READ – show all articles
    @GetMapping
    public String dashboard(HttpSession session, Model model) {
        if (!isLoggedIn(session)) return "redirect:/login";
        model.addAttribute("articles", articleService.findAll());
        model.addAttribute("adminUser",
            session.getAttribute(AuthController.SESSION_ADMIN_KEY));
        return "admin/dashboard";
    }

    // CREATE – show empty form
    @GetMapping("/new")
    public String showCreateForm(HttpSession session, Model model) {
        if (!isLoggedIn(session)) return "redirect:/login";
        model.addAttribute("article", new Article());
        model.addAttribute("mode", "create");
        return "admin/form";
    }

    // CREATE – save the new article
    @PostMapping
    public String createArticle(@Valid @ModelAttribute Article article,
```

```

BindingResult bindingResult, ...)

if (bindingResult.hasErrors()) { return "admin/form"; }

articleService.save(article);

return "redirect:/admin";
}

// UPDATE — show pre-filled edit form
@GetMapping("/edit/{id}")

// UPDATE — save the changes
@PostMapping("/edit/{id}")

article.setId(id); // <-- crucial: tells JPA to UPDATE not INSERT

articleService.save(article);

// DELETE — confirmation page
@GetMapping("/delete/{id}")

// DELETE — actually delete
@PostMapping("/delete/{id}")

articleService.deleteById(id);

return "redirect:/admin";
}

```

The guard pattern — isLoggedIn check

Every method starts with: `if (!isLoggedIn(session)) return 'redirect:/login';` This means if someone manually types `/admin/new` into the browser without logging in, they are immediately sent back to the login page. The session attribute is only set by `AuthController` after a successful login.

@Valid and BindingResult

`@Valid` triggers the `@NotBlank` validation on the `Article` object. `BindingResult` collects any errors. If `hasErrors()` is true, we return the form template again (with errors shown) instead of saving.

article.setId(id) before save for UPDATE

When the edit form is submitted, Spring creates a new `Article` object from the form data — but this new object has no id (so JPA would INSERT a duplicate). We call `article.setId(id)` — copying the id from the URL path — before saving. Now JPA sees an existing id and runs UPDATE instead of INSERT.

TIP: CRUD stands for Create, Read, Update, Delete. `AdminController` implements all four: GET `/admin/new` + POST `/admin` = Create; GET `/admin` = Read (list); GET + POST `/admin/edit/{id}` = Update; GET + POST `/admin/delete/{id}` = Delete.

17. Template: login.html

Thymeleaf templates are normal HTML files with special th: attributes. The browser never sees these attributes — Thymeleaf processes them on the server and sends plain HTML to the browser.

```
src/main/resources/templates/login.html (key parts)
```

```
<form th:action="@{/login}" th:object="${loginForm}" method="post">

<input type="text" th:field="*{username}" />

<input type="password" th:field="*{password}" />

<button type="submit">Log in</button>

</form>
```

Thymeleaf attribute	What it does
th:action="@{/login}"	Sets the form's action URL to /login (builds the correct URL even with a context path)
th:object="\${loginForm}"	Binds the whole form to the loginForm object placed in the Model by the Controller
th:field="*{username}"	Shortcut for name="username" id="username" value="\${loginForm.username}" — binds the field
th:href="@{/wiki}"	@{} always builds the correct URL — use it instead of hard-coding href values
th:text="\${value}"	Sets the text content of an element from the Model
th:if="\${condition}"	Renders the element only if the condition is true
th:unless="\${condition}"	Renders the element only if the condition is FALSE (opposite of th:if)
th:each="a : \${articles}"	Loops over the articles list — like a Java for-each loop
th:errors="*{title}"	Displays the @NotBlank error message for the title field

18. Template: wiki/list.html

The public wiki home page. Shows the article grid, search bar, category filter pills, and admin controls if the user is logged in.

templates/wiki/list.html (key sections)

```
<!-- Category filter pills -->

<div class="category-filters">

<a class="filter-pill" th:href="@{/wiki}">All</a>

<a class="filter-pill"
th:each="cat : ${categories}"
th:classappend="${category == cat} ? ' active'"
th:href="@{/wiki(category=${cat})}"
th:text="${cat}">Category</a>

</div>

<!-- Article cards -->

<article class="article-card" th:each="a : ${articles}">

<span class="badge" th:text="${a.category}"></span>

<h2><a th:href="@{'/wiki/' + ${a.id}}" th:text="${a.title}"></a></h2>

<p th:text="${#strings.abbreviate(a.content, 120)}"></p>

<p th:text="${#temporals.format(a.lastUpdated, 'dd MMM yyyy')}"></p>

<div class="card-actions" th:if="${isAdmin}">

<a class="btn small" th:href="@{'/admin/edit/' + ${a.id}}">Edit</a>

<a class="btn small danger" th:href="@{'/admin/delete/' + ${a.id}}">Delete</a>

</div>

</article>
```

th:classappend

Adds an extra CSS class to an element without removing existing classes. Here it adds ' active' to the pill whose category matches the current filter, making it appear highlighted in blue.

#strings.abbreviate(text, 120)

Thymeleaf's built-in utility. Cuts the content text at 120 characters and adds '...' at the end, so the article cards all have the same height.

#temporals.format(date, pattern)

Formats a `LocalDateTime` into a readable string like '03 Jun 2026'. Without this, the raw Java timestamp would show an ugly number.

19. Template: wiki/view.html

templates/wiki/view.html (key parts)

```
<!-- Admin controls – only shown when isAdmin is true -->
<div class="article-nav-row">
  <a class="back" th:href="@{/wiki}">&larr; Back to all articles</a>
  <div class="admin-actions" th:if="${isAdmin}">
    <a class="btn small" th:href="@{/admin/edit/' + ${article.id}}">Edit article</a>
    <a class="btn small danger" th:href="@{/admin/delete/' + ${article.id}}">Delete
    article</a>
  </div>
</div>

<article class="article-full">
  <span class="badge" th:text="${article.category}"></span>
  <h1 th:text="${article.title}"></h1>
  <div class="content" th:text="${article.content}"></div>
</article>
```

Why th:text instead of th:utext for content?

th:text auto-escapes HTML special characters (< becomes < etc.). This means if someone puts <script> tags in an article, the browser displays them as plain text instead of running them. This protects against Cross-Site Scripting (XSS) attacks. th:utext would render raw HTML — only use it when you fully trust the content.

20. Template: admin/dashboard.html

templates/admin/dashboard.html (key parts)

```
<nav class="topbar admin">
  <span>Logged in as <strong th:text="${adminUser}"></strong></span>
  <a th:href="@{/wiki}">View public wiki</a>
  <a th:href="@{/logout}">Log out</a>
</nav>

<!-- th:unless hides the table when there are no articles -->
<table class="data-table" th:unless="${#lists.isEmpty(articles)}">
  <tr th:each="a : ${articles}">
    <td th:text="${a.id}"></td>
    <td><a th:href="@{'/wiki/' + ${a.id}}" th:text="${a.title}"></a></td>
    <td th:text="${#temporals.format(a.lastUpdated, 'dd MMM yyyy HH:mm')}"></td>
    <td>
      <a th:href="@{'/admin/edit/' + ${a.id}}">Edit</a>
      <a th:href="@{'/admin/delete/' + ${a.id}}">Delete</a>
    </td>
  </tr>
</table>
```

#lists.isEmpty(articles)

Thymeleaf utility method. Returns true if the list has zero items. Used here to show a friendly 'No articles yet' message instead of an empty table.

21. Template: admin/form.html

One template that handles BOTH create and edit. The 'mode' attribute passed by the Controller controls the heading text, the form action URL, and the button label.

```
templates/admin/form.html (key parts)

<!-- Title switches based on mode -->

<h1 th:text="${mode == 'edit'} ? 'Edit article' : 'New article'"></h1>

<!-- Form action switches too -->

<form th:action="${mode == 'edit'} ? @{'/admin/edit/' + ${article.id}} : @{/admin}"
th:object="${article}" method="post">

<label for="title">Title</label>

<input type="text" id="title" th:field="*{title}"/>

<p class="error-text" th:if="${#fields.hasErrors('title')}"
th:errors="*{title}"></p>

<label for="category">Category</label>

<input type="text" id="category" th:field="*{category}"/>

<label for="content">Content</label>

<textarea id="content" th:field="*{content}" rows="10"></textarea>

<a class="btn secondary" th:href="@{/admin}">Cancel</a>

<button class="btn primary" type="submit"
th:text="${mode == 'edit'} ? 'Save changes' : 'Create article'"></button>

</form>
```

The ternary operator in Thymeleaf — ? :

`${mode == 'edit'} ? 'Edit article' : 'New article'` means: if mode equals 'edit', use 'Edit article'; otherwise use 'New article'. Same as the Java ternary operator (condition ? valueIfTrue : valueIfFalse).

`#fields.hasErrors('title')` and `th:errors`

`#fields.hasErrors` checks whether the BindingResult has an error for a specific field. `th:errors` displays the error message text (e.g. 'Title is required'). These only appear after a failed form submission.

22. Template: admin/confirm-delete.html

```
templates/admin/confirm-delete.html (key parts)

<div class="card error">

<h1>Delete article?</h1>

<blockquote>

<strong th:text="${article.title}"></strong>

<span class="badge" th:text="${article.category}"></span>

</blockquote>

<p>This action cannot be undone.</p>

<form th:action="@{'/admin/delete/' + ${article.id}}" method="post">

<a class="btn secondary" th:href="@{/admin}">Cancel</a>

<button class="btn danger" type="submit">Yes, delete</button>

</form>

</div>
```

Why use a POST form for delete instead of just a link?

A normal link (anchor tag) sends a GET request. GET requests should never change data — they are meant for reading. If delete were a GET link, a browser prefetch, a search-engine crawler, or someone accidentally opening the link could delete an article without confirmation. Using a POST form means the browser only sends the delete request when the user actually clicks the 'Yes, delete' button. This is called the Post-Redirect-Get (PRG) pattern.

23. Viva Q&A; — 30 Likely Questions

Read each answer aloud at least once before the viva. Use your own words — do not memorise word for word.

Q: What does `@SpringBootApplication` do?

A: It does three things: (1) enables auto-configuration so Spring sets up the database and Thymeleaf automatically, (2) triggers component scanning so Spring finds all my `@Controller`, `@Service` and `@Repository` classes, and (3) marks the class as a configuration source.

Q: What is MVC and how does your project use it?

A: MVC is Model-View-Controller. The Model is my Article and Admin classes (data). The View is my Thymeleaf HTML templates. The Controller layer handles HTTP requests, calls the Service for data, puts it in the Model, and returns the template name. This separation means I can change the HTML without touching the Java code.

Q: What is the difference between `@GetMapping` and `@PostMapping`?

A: `@GetMapping` handles GET requests — when the browser navigates to a URL or clicks a link. I use it to show forms and pages. `@PostMapping` handles POST requests — when a form is submitted. I use it to process login, create articles and delete articles.

Q: What is Thymeleaf?

A: Thymeleaf is a Java template engine. I write normal HTML files with extra `th:` attributes. When a request comes in, Thymeleaf processes the template on the server, replaces all the `th:` attributes with real values from the Model, and sends plain HTML to the browser. The browser never sees any Thymeleaf code.

Q: What is a JPA entity?

A: A JPA entity is a Java class annotated with `@Entity` that is automatically mapped to a database table. Each field becomes a column. Each instance of the class becomes a row. In my project, Article maps to the ARTICLE table and Admin maps to the ADMIN table.

Q: What does JpaRepository give you?

A: By extending JpaRepository, I automatically get findAll(), findById(), save(), deleteById() and existsById() without writing any code. Spring generates the SQL queries at startup. I can also declare custom methods just by naming them correctly, like findByCategoryIgnoreCase().

Q: How does the login work end to end?

A: The user submits the login form. The browser sends a POST to /login. AuthController receives it, calls AuthService.isAuthenticated(). AuthService looks up the username in the ADMIN table using AdminRepository. If found and the password matches, AuthController stores the username in the HttpSession and redirects to /admin. If not, it shows the error page.

Q: What is an HttpSession and how do you use it?

A: HttpSession is server-side storage tied to one browser. Spring gives each browser a unique cookie. When I call session.setAttribute('loggedInAdmin', username), the username is stored on the server linked to that cookie. On every admin page, I check session.getAttribute('loggedInAdmin') — if it is null, the user is not logged in and I redirect them to /login. session.invalidate() on logout deletes everything.

Q: How do you protect the admin pages?

A: Every method in AdminController starts with: if (!isLoggedIn(session)) return 'redirect:/login'; The isLoggedIn() helper checks whether the session has the 'loggedInAdmin' key. If not, the user is sent to the login page regardless of what URL they typed.

Q: What is the difference between @NotBlank on LoginForm and @NotBlank on Article?

A: On LoginForm, @NotBlank prevents the login form from being submitted empty. The Controller uses @ModelAttribute (not @Valid), so AuthService does its own null checks. On Article, @NotBlank triggers Spring's validation when @Valid is used in AdminController. If any field is blank, BindingResult captures the error and the form is re-displayed with the error message shown next to the field.

Q: What does Optional mean?

A: Optional is a wrapper that either contains a value or is empty — it is never null. When findById() cannot find an article, it returns an empty Optional instead of null. I call maybeArticle.isEmpty() to check if nothing was found, and maybeArticle.get() to get the actual article. This avoids NullPointerExceptions.

Q: How does CREATE work in AdminController?

A: GET /admin/new shows an empty form (a new Article() with no id). POST /admin receives the submitted form, validates it with @Valid, and calls articleService.save(article). Since the article has no id, JPA runs INSERT INTO article ... and assigns a new id automatically.

Q: How does UPDATE differ from CREATE?

A: GET /admin/edit/{id} loads the existing article from the database and pre-fills the form. When the form is submitted (POST /admin/edit/{id}), Spring creates a new Article from the form data — but this new object has no id. I call article.setId(id) to set the id from the URL, then save(). JPA sees an existing id and runs UPDATE instead of INSERT.

Q: Why do you use a POST form for delete instead of a link?

A: A link sends a GET request. GET should never change data. A browser might prefetch links or a search engine might crawl them, accidentally deleting articles. Using a POST form means the delete only happens when the user physically clicks the 'Yes, delete' button on the confirmation page.

Q: What is the Post-Redirect-Get pattern?

A: After processing a POST request (create, update, delete), the controller returns 'redirect:/admin' instead of rendering a template directly. This tells the browser to make a new GET request to /admin. If the user refreshes the page, the GET is repeated — not the POST. Without this, refreshing would re-submit the form and create duplicate articles.

Q: What is H2 and why did you use it?

A: H2 is a small SQL database written in Java. It runs inside the application process — no installation needed. I configured it to store data in a file (./data/wikidb) so articles survive server restarts. It also has a browser console at /h2-console which is useful for showing the database contents during a demo.

Q: What does ddl-auto=update mean?

A: It tells Hibernate (the JPA implementation) to automatically create the ARTICLE and ADMIN tables if they do not exist, and update the schema if I add a new field to an entity class. It never drops existing data. I use this during development; in production you would use 'validate' or 'none'.

Q: What is data.sql and why does it use MERGE?

A: data.sql is a SQL file that Spring runs automatically every startup to seed the database with an admin account and sample articles. I use MERGE instead of INSERT because the file runs on every startup. INSERT would fail the second time because the rows already exist. MERGE says: insert if not there, skip if already there.

Q: What is constructor injection and why is it preferred?

A: Constructor injection is when Spring passes the dependencies (like ArticleRepository) through the constructor. This is preferred over field injection (@Autowired on a field) because: the dependency cannot be null (it is required at construction time), the class can be tested by just passing a mock in the constructor, and it makes dependencies explicit and visible.

Q: What does @RequestParam do?

A: @RequestParam extracts a value from the URL query string. For example, /wiki?search=java passes 'java' to the search parameter. I use required=false so the parameter is optional — if the URL has no ?search=..., Spring passes null and the controller shows all articles.

Q: What does @PathVariable do?

A: @PathVariable extracts a value from the URL path. In @GetMapping('/{id}'), the {id} is a placeholder. When someone visits /wiki/3, Spring extracts '3', converts it to a Long, and passes it to the id parameter.

Q: What is the Model in Spring MVC?

A: The Model is a container (like a Map) that the Controller uses to pass data to the template. model.addAttribute('articles', list) puts the list in the Model with the key 'articles'. In the Thymeleaf template, \${articles} reads it back. The Model only exists for the duration of one request.

Q: How do you build the categories list in WikiController?

A: I use a simple for loop. I start with an empty ArrayList. I loop through every article using a for-each loop. Inside the loop I check: if the article's category is not already in the list, I add it. After the loop I call Collections.sort() to sort the list alphabetically. This produces ['General', 'History', 'Programming', 'Testing'] which the template displays as clickable filter pills.

Q: What is the layered architecture and why is it important?

A: My project has three layers: Controller, Service, Repository. The Controller only handles HTTP — it does not write SQL. The Repository only talks to the database — it does not know about HTTP. The Service holds the business logic in between. This means I can change the database (e.g. swap H2 for MySQL) by only changing the Repository, without touching the Controller or templates.

Q: What is the difference between Part A and Part B?

A: Part A: login with hard-coded credentials (no database), only the login flow. Part B: H2 database, JPA entities (Article, Admin), Spring Data repositories, full CRUD for articles via AdminController, public wiki via WikiController, and HttpSession-based authentication instead of hard-coded checks.

Q: How does the category filter work?

A: WikiController loops through all articles and collects unique category names into an ArrayList. It sorts the list and puts it in the Model as 'categories'. The template shows each as a clickable pill link: /wiki?category=Programming. When clicked, the browser sends a GET with ?category=Programming, the Controller reads it with @RequestParam and calls articleService.findByCategory(category), which runs `SELECT ... WHERE UPPER(category) = UPPER(?)`.

Q: What is isAdmin and how do templates use it?

A: WikiController checks `session.getAttribute('loggedInAdmin')` and puts a boolean `isAdmin = true/false` in the Model. The template uses `th:if='${isAdmin}'` to show the Edit/Delete buttons and the admin nav links only when `isAdmin` is true. This is the correct MVC approach — the Controller decides, the template just displays.

Q: What happens if I try to edit an article that does not exist?

A: AdminController calls `articleService.findById(id)` which returns an Optional. If the Optional is empty (no such id), the controller adds an errorMessage to the Model and returns the 'error' template, which shows a friendly message. It does not crash with an exception.

Q: What is the @Service annotation?

A: @Service marks a class as a Spring-managed service bean. Spring creates one instance of it when the application starts and injects it wherever it is needed (e.g. into AuthController and AdminController). Without this annotation, Spring would not find the class during component scanning.

Q: How would you add a second admin user?

A: I would add another `MERGE INTO ADMIN ...` row in `data.sql` with a different id and username. When the application next starts, `data.sql` runs and inserts the new row. The `AdminRepository.findByUsername()` query will then find this user when they log in. No Java code changes needed — that is the benefit of using a database instead of hard-coded credentials.

FINAL TIP: Final viva tip: if you get a question you are not sure about, do not panic. Say: 'Let me walk you through the code'. Open the relevant file in your IDE, point to the lines, and explain what each annotation or method does. Examiners value understanding over memorisation.